Pasos para Analizar Imágenes fMRI

https://gemini.google.com/app/42a703d0709f2744

User prompt: Pasos para analizar imágenes de resonancia magnética funcional

Response: El análisis de imágenes de resonancia magnética funcional (fMRI) sigue un flujo de trabajo estructurado para transformar la señal BOLD (*Blood Oxygenation Level Dependent*) en mapas estadísticos de activación o conectividad neurobiológicamente interpretables.

1 Organización y Conversión de Datos

Estandarización del formato de entrada

- **Conversión DICOM a NIfTI:** Transformación de las imágenes brutas del escáner (`.dcm`) a formato `.nii` o `.nii.gz`.
- **Estructuración BIDS:** Organización de los archivos bajo el estándar *Brain Imaging Data Structure* (BIDS) para automatizar el procesamiento en paquetes como fMRIPrep, SPM, FSL o AFNI.

2 Preprocesamiento

Corrección de artefactos físicos y alineación

- **Corrección de tiempo de corte (*Slice Timing Correction*):** Ajuste temporal para compensar que los cortes del volumen funcional no se adquieren simultáneamente.
- **Realineación / Corrección de movimiento:** Registro rígido (6 grados de libertad) para corregir el movimiento de la cabeza entre volúmenes. Se generan los parámetros de traslación y rotación (x, y, z).
- **Co-registro:** Alineación de las imágenes funcionales (EPI) con la imagen anatómica de alta resolución (T1w) del mismo sujeto.
- **Segmentación y Normalización Espacial:** Deformación no lineal de la anatomía del sujeto a un espacio estereotáxico estándar (ej. MNI152 o Talairach) para permitir comparaciones inter-sujeto.
- **Suavizado Espacial (*Spatial Smoothing*):** Convolución con un filtro gaussiano (FWHM típicamente de 4 a 8 mm) para aumentar la relación señal-ruido y adaptar los datos a los supuestos de la teoría de campos aleatorios.

3 Modelado Estadístico de Primer Nivel

Estimación a nivel de sujeto individual

- **Filtrado temporal:** Aplicación de un filtro pasa-altas (ej. corte en 128 s) para eliminar derivas de baja frecuencia e interferencias scanner/fisiológicas.
- **Modelado Lineal General (GLM):** Convolución del diseño experimental (bloques o eventos) con la función de respuesta hemodinámica (*HRF*).
- **Regresores de confusión:** Inclusión de parámetros de movimiento, señales de líquido cefalorraquídeo/sustancia blanca o matrices de regresión de ruido (ej. ACompCor).
- **Estimación de contrastes:** Generación de mapas de coeficientes (β) y estadísticas (t o Z) para las condiciones de interés.

4 Análisis de Segundo Nivel o Grupal

Inferencia poblacional y conectividad

- **Estadística grupal:** Combinación de los mapas de contraste individuales mediante modelos de efectos mixtos o efectos aleatorios (Random Effects, RFX) para generalizar resultados a la población.
- **Análisis de Conectividad (si aplica):**
  - *Basado en semilla (Seed-based):* Correlación de la serie temporal de un ROI con el resto del cerebro.
  - *Análisis de Componentes Independientes (ICA):* Identificación de redes de estado de reposo (*Resting State Networks* como la DMN).

5 Corrección por Múltiples Comparaciones y Visualización

Control de falsos positivos

- **Ajuste de umbral:** Aplicación de correcciones por *Family-Wise Error Rate* (FWE) o *False Discovery Rate* (FDR) a nivel de vóxel o clúster para controlar el problema de comparaciones múltiples ($> 100,000$ vóxeles).
- **Visualización:** Superposición de las áreas estadísticamente significativas sobre plantillas anatómicas estructurales o proyecciones corticales en superficie (ej. FreeSurfer/Connectome Workbench).

User prompt: Muestra un código ejecutable en Python usando Nilearn para hacer el preprocesamiento y GLM de primer nivel en fMRI.

Response: El siguiente script utiliza **Nilearn** para descargar un dataset de prueba (*Development FSL dataset*), ejecutar el modelado lineal

general (GLM) de primer nivel directamente sobre las imágenes funcionales y extraer los mapas estadísticos de activación.

Nilearn abstrae el preprocesamiento básico (como el suavizado espacial y el filtrado temporal) integrándolo en el ajuste del modelo mediante la clase `FirstLevelModel`.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from nilearn import datasets
from nilearn.glm.first_level import FirstLevelModel
from nilearn.reporting import make_glm_report
from nilearn.plotting import plot_design_matrix, plot_stat_map

# ============================================================================
# 1. CARGA DE DATOS DE EJEMPLO
# ============================================================================
# Descargamos los datos de un sujeto de desarrollo (tarea de observación de videos)
data = datasets.fetch_development_fmri(n_subjects=1)
fmri_img = data.func[0]        # Archivo NIfTI funcional (4D)
events_file = data.events[0]  # Archivo TSV con marcas de tiempo de los eventos
confounds_file = data.confounds[0] # Confounds (movimiento, etc.)

# Tiempo de repetición (TR) del dataset
t_r = 2.0

# Cargar eventos
events = pd.read_csv(events_file, sep='\t')
print("Primeras filas del archivo de eventos:")
print(events.head())

# ============================================================================
# 2. DEFINICIÓN DEL MODELO DE PRIMER NIVEL (PREPROCESAMIENTO + GLM)
# ============================================================================
# FirstLevelModel aplica internamente el suavizado, filtrado y convolución HRF
fmri_glm = FirstLevelModel(
    t_r=t_r,
    hrf_model='glover',           # Función de respuesta hemodinámica canónica
    drift_model='cosine',         # Filtrado de bajas frecuencias (pasa-altas)
    high_pass=0.01,               # Frecuencia de corte en Hz
    smoothing_fwhm=6.0,           # Suavizado espacial con filtro gaussiano de 6mm
    noise_model='ar1',            # Modelo autoregresivo AR(1) para autocorrelación
    standardize=True,             # Estandarización de las señales
    minimize_memory=True
)

# ============================================================================
# 3. AJUSTE DEL MODELO (FIT)
# ============================================================================
# Ajustamos el GLM incluyendo los confounds de movimiento
fmri_glm = fmri_glm.fit(fmri_img, events=events, confounds=confounds_file)

# Inspección de la matriz de diseño
design_matrix = fmri_glm.design_matrices_[0]
fig, ax = plt.subplots(figsize=(6, 8))
plot_design_matrix(design_matrix, ax=ax)
plt.title("Matriz de Diseño del GLM", fontsize=12)
plt.show()

# ============================================================================
# 4. DEFINICIÓN Y CÁLCULO DE CONTRASTES
# ============================================================================
# Creamos un vector de contraste para evaluar la condición de interés vs. línea base
# Suponiendo que el evento se llama 'show' en la matriz de diseño:
contrast_matrix = np.zeros(design_matrix.shape[1])
if 'show' in design_matrix.columns:
    contrast_matrix[design_matrix.columns == 'show'] = 1
else:
    # Si la columna tiene otro nombre, tomamos la primera condición
    contrast_matrix[0] = 1
```

```
# Calcular la estimación estadística (mapa de z-scores)
z_map = fmri_glm.compute_contrast(contrast_matrix, output_type='z_score')

# ============================================================================
# 5. VISUALIZACIÓN DE RESULTADOS
# ============================================================================
# Graficar el mapa estadístico superpuesto en la plantilla anatómicamente estándar
plot_stat_map(
    z_map,
    threshold=3.1,              # Umbral de activación (z > 3.1 equivale a p < 0.001)
    display_mode='z',           # Cortes axiales
    cut_coords=5,               # 5 cortes
    black_bg=True,
    title='Mapa de Activación Z-score (Umbral p < 0.001)'
)
plot_stat_map(
    z_map,
    threshold=3.1,
    display_mode='ortho',       # Vista ortogonal (sagital, coronal, axial)
    title='Vista Ortogonal de la Activación'
)
plt.show()

# ============================================================================
# 6. GENERACIÓN DE REPORTE HTML HTML COMPLETO
# ============================================================================
# Crea un reporte interactivo con diagnósticos, calidad del ajuste y mapas
report = make_glm_report(
    fmri_glm,
    contrasts={'Condición_Interés': contrast_matrix},
    title='Reporte GLM Primer Nivel'
)
report.save_as_html('reporte_glm_primer_nivel.html')
print("Reporte interactivo guardado como 'reporte_glm_primer_nivel.html'")
```

Requisitos previos para ejecutar

Para instalar las dependencias necesarias en tu entorno de Python:

```
pip install nilearn pandas matplotlib scipy
```

User prompt: ¿Cómo modifico el script de Nilearn para procesar un dataset organizado en formato BIDS automáticamente?

Response: Para procesar automáticamente un conjunto de datos organizado según el estándar **BIDS** (*Brain Imaging Data Structure*), Nilearn proporciona la clase `first_level_from_bids`.

Esta función escanea la estructura de carpetas de BIDS, localiza los archivos NIfTI funcionales, lee automáticamente los archivos de eventos (`_events.tsv`) y extrae los regresores de confusión (*confounds*) si el dataset ya fue preprocesado por herramientas como **fMRIPrep**.

```
import numpy as np
import matplotlib.pyplot as plt
from nilearn.glm.first_level import first_level_from_bids, FirstLevelModel
from nilearn.plotting import plot_design_matrix, plot_stat_map

# ============================================================================
# 1. RUTA AL DATASET EN FORMATO BIDS
# ============================================================================
# Directorio raíz del dataset BIDS (o la salida de fMRIPrep en 'derivatives')
bids_dir = '/ruta/a/tu/dataset_bids'

# Etiqueta de la tarea experimental tal como aparece en las carpetas (ej. task-rest, task-pinger)
task_label = 'pinger'

# ============================================================================
# 2. GENERACIÓN AUTOMÁTICA DE MODELOS DESDE BIDS
# ============================================================================
# 'first_level_from_bids' analiza el directorio y construye la lista de modelos,
# asociando automáticamente cada NIfTI con sus eventos y confounds correspondientes.
```

```
models, models_run_imgs, models_events, models_confounds = first_level_from_bids(
    dataset_path=bids_dir,
    task_label=task_label,
    space_label='MNI152NLin2009cAsym',  # Espacio de normalización generado por fMRIPrep
    smoothing_fwhm=6.0,                  # Suavizado gaussiano en mm
    derivatives_folder='derivatives',    # Carpeta donde fMRIPrep guardó los preprocesados
    hrf_model='glover',
    drift_model='cosine',
    high_pass=0.01,
    minimize_memory=True,
    n_jobs=-1                            # Parateltrización para usar todos los núcleos CPU
)

# ==============================================================================
# 3. AJUSTE DEL MODELO (FIT) Y CÁLCULO DE CONTRASTES
# ==============================================================================
# Iteramos sobre los sujetos y corridas encontradas
for model, run_imgs, events, confounds in zip(models, models_run_imgs, models_events, models_confounds):

    # Identificar sujeto desde la estructura del objeto
    subject_id = model.subject_label
    print(f"\n--- Procesando Sujeto: sub-{subject_id} ---")

    # Ajustar el GLM para este sujeto usando sus datos asociados
    model.fit(run_imgs, events=events, confounds=confounds)

    # Graficar matriz de diseño del sujeto
    design_matrix = model.design_matrices_[0]
    fig, ax = plt.subplots(figsize=(6, 8))
    plot_design_matrix(design_matrix, ax=ax)
    plt.title(f"Matriz de Diseño - sub-{subject_id}", fontsize=12)
    plt.show()

    # ==============================================================================
    # 4. ESTIMACIÓN DE CONTRASTES
    # ==============================================================================
    # Puedes definir contrastes usando fórmulas de texto que referencien
    # los nombres de las columnas en la matriz de diseño (extraídos del _events.tsv)

    # Ejemplo: si el TSV de eventos tiene una condición llamada 'estimulo_visual'
    columnas = design_matrix.columns
    print(f"Columnas detectadas en el GLM: {list(columnas)}")

    if 'estimulo_visual' in columnas:
        contrast_formula = 'estimulo_visual'
    else:
        # Alternativa: contraste contra el promedio del resto si hay varias condiciones
        contrast_formula = columnas[0]  # Evaluamos la primera condición detectada

    print(f"Calculando contraste para: '{contrast_formula}'")
    z_map = model.compute_contrast(contrast_formula, output_type='z_score')

    # ==============================================================================
    # 5. VISUALIZACIÓN
    # ==============================================================================
    plot_stat_map(
        z_map,
        threshold=3.1,
        display_mode='z',
        cut_coords=5,
        title=f"Activación Z-score: sub-{subject_id} ({contrast_formula})"
    )
    plt.show()
```

Aspectos clave al trabajar con BIDS y Nilearn

1. **Requisito de preprocesamiento:** `first_level_from_bids` asume que tus datos ya han pasado por un flujo de preprocesamiento anatómico y funcional (como **fMRIPrep**) ubicado dentro de la carpeta `derivatives/`.
2. **Nombres de archivos normalizados:** Los archivos de eventos deben seguir estrictamente la convención `sub-<label>_task-`

`<label>_events.tsv` para que Nilearn pueda asociarlos automáticamente a sus volúmenes de fMRI (`.nii.gz`).

3. **Selección de regresores de confusión:** Por defecto, Nilearn extraerá los parámetros de movimiento y métricas de calidad de los archivos `*_desc-confounds_timeseries.tsv` generados por fMRIPrep.

User prompt: Muestra cómo tomar los mapas de primer nivel de varios sujetos y hacer un análisis GLM de segundo nivel (grupal) en Nilearn.

Response: Para realizar un análisis estadístico de **segundo nivel (grupal)** en Nilearn, se combinan los mapas de contraste ajustados de primer nivel de cada sujeto. En este nivel, la unidad de observación es el sujeto, y se evalúa si una activación observada es consistente a nivel poblacional (efectos aleatorios / *Random Effects*).

El siguiente script muestra cómo calcular los mapas de primer nivel para múltiples sujetos mediante `first_level_from_bids` (o iteración individual) y enviarlos a la clase `SecondLevelModel` de Nilearn.

```
import pandas as pd
import matplotlib.pyplot as plt
from nilearn.glm.first_level import first_level_from_bids
from nilearn.glm.second_level import SecondLevelModel
from nilearn.plotting import plot_design_matrix, plot_stat_map
from nilearn.glm import threshold_stats_img

# ==============================================================================
# 1. GENERACIÓN/CARGA DE MAPAS DE PRIMER NIVEL (NIVEL INDIVIDUAL)
# ==============================================================================
bids_dir = '/ruta/a/tu/dataset_bids'
task_label = 'pinger'
contrast_of_interest = 'estimulo_visual'  # Nombre de la condición en events.tsv

# Inicializamos modelos de BIDS para todos los sujetos
models, models_run_imgs, models_events, models_confounds = first_level_from_bids(
    dataset_path=bids_dir,
    task_label=task_label,
    space_label='MNI152NLin2009cAsym',
    smoothing_fwhm=6.0,
    derivatives_folder='derivatives',
    minimize_memory=True,
    n_jobs=-1
)

# Almacenaremos los mapas estadísticos (efectos o z-scores) y las etiquetas de sujeto
cmap_maps = []
subject_ids = []

print("Calculando contrastes de primer nivel por sujeto...")
for model, run_imgs, events, confounds in zip(models, models_run_imgs, models_events, models_confounds):
    sub_id = model.subject_label

    # Ajustar GLM individual
    model.fit(run_imgs, events=events, confounds=confounds)

    # Calcular el mapa de tamaño del efecto ('effect_size') o mapa 'z_score'
    # 'effect_size' se recomienda cuando se pasa como entrada a un GLM de 2º nivel
    effect_map = model.compute_contrast(contrast_of_interest, output_type='effect_size')

    cmap_maps.append(effect_map)
    subject_ids.append(f"sub-{sub_id}")

print(f"Total de sujetos procesados para el 2º nivel: {len(subject_ids)}")

# ==============================================================================
# 2. CONSTRUCCIÓN DE LA MATRIZ DE DISEÑO DEL SEGUNDO NIVEL
# ==============================================================================
# Para una prueba t de una muestra (One-sample t-test: H0: activación media = 0)
# Creamos un DataFrame donde cada fila representa a un sujeto y hay una columna intercepción (1s)
design_matrix_2nd = pd.DataFrame(
    {'intercept': [1.0] * len(subject_ids)},
    index=subject_ids
)
```

```
# Visualizar la matriz de diseño de 2º nivel
fig, ax = plt.subplots(figsize=(4, 6))
plot_design_matrix(design_matrix_2nd, ax=ax)
plt.title("Matriz de Diseño (2º Nivel: One-sample t-test)")
plt.show()

# ============================================================================
# 3. DEFINICIÓN Y AJUSTE DEL MODELO DE SEGUNDO NIVEL
# ============================================================================
second_level_model = SecondLevelModel(
    smoothing_fwhm=None,  # El suavizado ya se aplicó en el 1er nivel
    minimize_memory=True,
    n_jobs=-1
)

# Ajustar el modelo usando la lista de mapas NIfTI y la matriz de diseño poblacional
second_level_model.fit(cmap_maps, design_matrix=design_matrix_2nd)

# Evaluar el contraste (probar si la columna 'intercept' es mayor que 0)
z_map_group = second_level_model.compute_contrast(
    second_level_contrast='intercept',
    output_type='z_score'
)

# ============================================================================
# 4. CORRECCIÓN POR MÚLTIPLES COMPARACIONES Y UMBRALIZACIÓN
# ============================================================================
# Aplicamos control de False Discovery Rate (FDR) o Family-Wise Error (FWE)
thresholded_map, threshold = threshold_stats_img(
    z_map_group,
    alpha=0.05,
    height_control='fdr',     # Opciones: 'fpr', 'fdr', 'bonferroni'
    cluster_threshold=10      # Tamaño mínimo del clúster en vóxeles
)

print(f"Umbral Z equivalente ajustado por FDR (p < 0.05): Z = {threshold:.3f}")

# ============================================================================
# 5. VISUALIZACIÓN DE RESULTADOS GRUPALES
# ============================================================================
# Mapa z sin corregir vs. umbralizado
plot_stat_map(
    z_map_group,
    threshold=3.1,
    display_mode='z',
    cut_coords=5,
    title='Mapa Grupal Sin Corregir (Z > 3.1)'
)

plot_stat_map(
    thresholded_map,
    threshold=threshold,
    display_mode='z',
    cut_coords=5,
    title='Mapa Grupal Corregido (FDR p < 0.05, K >= 10)'
)
plt.show()
```

Puntos clave para el análisis poblacional

1. **Entrada al 2º Nivel (`effect_size` vs `z_score`):**
    - Es computacionalmente y estadísticamente preferible pasar los mapas de **`effect_size`** (β) o **`stat`** (t) del primer nivel en lugar de los mapas Z, ya que preservan la escala del cambio en la señal BOLD entre sujetos.
2. **Diseño experimental poblacional:**
    - **One-sample t-test:** Para evaluar la activación general de un grupo (`intercept` con columna de 1s).
    - **Two-sample t-test / ANOVA:** Si comparas dos grupos de sujetos (ej. Controles vs. Pacientes), la matriz de diseño contendrá dos

columnas categóricas e indicadores $0/1$.

- **Regresión/Covaríables:** Puedes añadir columnas adicionales al DataFrame `design_matrix_2nd` con variables continuas por sujeto (ej. Edad, Puntuación de desempeño) para analizar correlaciones conductuales.
